

Construcción y adaptación de payloads

Objetivos de aprendizaje

Construir una prueba desde el contexto, justificar cada componente, seleccionar evidencia in-band o blind y adaptar una restricción sin recurrir a listas ciegas.

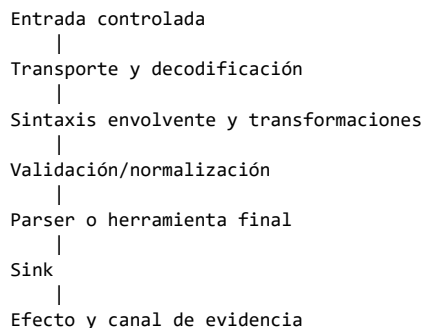
Prerrequisitos

HTTP, formatos de cuerpo, URL, encoding, parsers, flujo source-sink, shell, argumentos, SQL y permisos.

Por qué no memorizar cadenas

Un payload es una hipótesis serializada para un parser concreto. La misma cadena puede ser código en una shell, texto en argv, un valor JSON inválido o datos escapados en SQL. Memorizar oculta cinco preguntas: dónde se inserta, qué sintaxis existe alrededor, qué transformaciones ocurren, qué componente interpreta y qué evidencia puede observarse.

Modelo mental



Método de construcción

1. Copia una entrada válida.
2. Marca el punto exacto de inserción.
3. Escribe la sintaxis antes y después del valor.
4. Define una primitiva mínima: cambiar booleano, leer archivo conocido, ejecutar id, provocar una petición a un listener propio o alterar un argumento inocuo.
5. Decide si necesitas cerrar sintaxis y si debes reabrirlo.
6. Serializa para la capa exterior: URL, formulario, JSON, XML o multipart.
7. Predice respuesta y control negativo.
8. Ejecuta una sola variación.
9. Clasifica el fallo antes de adaptar.

Contextos de interpretación

Contexto	Unidad estructural	Control básico
SQL	token, literal, expresión, cláusula	consulta parametrizada
Shell	palabra, operador, expansión, redirección	ejecución sin shell y argv fijo
Argumentos	elemento de argv, opción, operando	--, allowlist y API segura
Ruta	segmento, raíz, enlace, ruta canónica	identificador indirecto y canonicalización
URL/SSRF	esquema, autoridad, host, puerto, path	parser único y política sobre destino efectivo
Plantilla	texto, expresión, sentencia	no evaluar plantillas controlables
Upload	metadatos, bytes, ubicación, servidor posterior	almacenamiento aislado y nombre generado
Autenticación	identidad, factor, estado, transición	servidor como autoridad de estado

Cierre, reapertura y separadores

Si el valor se inserta dentro de un literal SQL, la comilla puede cerrar ese literal; después se aporta una expresión y se neutraliza o reconstruye el sufijo. Si se inserta como número, una comilla puede ser innecesaria y romper la prueba. En shell, ;, &&, | y salto de línea solo son separadores si una shell los analiza. En argv pueden ser caracteres ordinarios.

Quoting pertenece a una capa. Las comillas que preservan JSON no son automáticamente las que necesita Bash o SQL. Dibuja de fuera a dentro:

```
PowerShell/curl -> JSON -> valor -> shell remota
```

Escapa cada capa una vez y verifica los bytes recibidos cuando el laboratorio lo permita.

Encoding y normalización

Codifica para transportar caracteres reservados; no lo uses como sinónimo de evasión. Para estudiar un filtro, registra:

- forma enviada;
- forma tras cada decodificación;
- comparación aplicada;
- forma que alcanza el sink;
- número de decodificaciones.

Un bypass solo está explicado cuando se identifica una discrepancia concreta entre validación y consumo.

Argumentos y opciones

Command injection introduce gramática de shell. Argument injection consigue elementos adicionales de argv. Option injection hace que un operando sea interpretado como opción. Prueba mínima: lograr una opción de salida inocua o un error inequívoco de la herramienta, no ejecutar código de inmediato. Consulta --help/manual de la versión detectada; -- no es universal.

Protocolos alternativos

En SSRF o clientes de URL, un esquema alternativo solo importa si la biblioteca lo soporta y el destino ofrece ese protocolo. Primero demuestra una petición HTTP a un listener propio; después verifica esquemas permitidos. No infieras soporte de `file:`, `gopher:` u otros por conocerlos de memoria.

Blacklist y allowlist

Una blacklist compara representaciones peligrosas conocidas y suele fallar frente a formas equivalentes. Una allowlist correcta expresa la propiedad permitida y se aplica tras parsing/canonicalización. Incluso una allowlist de host debe comprobar IP resuelta, puerto, redirecciones y cambios de resolución según el modelo de amenaza.

Canales de evidencia

- In-band: el resultado aparece en la respuesta.
- Booleano: dos condiciones producen respuestas distinguibles.
- Error: un error controlado revela parser o datos.
- Temporal: una demora depende de la condición y se compara con varias muestras.
- OOB/callback: el servidor contacta un endpoint HTTP o DNS propio del laboratorio.

Para pruebas blind usa controles negativos y repetición. Un retraso aislado o una resolución DNS no atribuible no confirma la primitiva completa.

Diagnóstico general

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
400 inmediato	transporte o formato inválido	enviar el original y cambiar un byte	corregir serialización
misma respuesta	entrada ignorada, caché o evidencia invisible	marcador único y control negativo	buscar otro canal
error de parser	contexto/sintaxis incorrectos	reducir a delimitador mínimo	ajustar cierre/reapertura
filtro explícito	comparación textual	comparar formas antes/después	corregir propiedad o estudiar normalización
efecto parcial	sink alcanzado con restricciones	primitiva más pequeña	adaptar a permisos/capacidad
comportamiento inestable	tiempo, estado o balanceo	repetir y fijar sesión	medir distribución

Árbol de diagnóstico general

```

INPUT DEL USUARIO
|
+-- ¿La petición exterior es válida? -- no --> reparar HTTP/JSON/form/multipart
|
+-- ¿La entrada llega a la función? -- no --> revisar nombre, ruta, estado y precondition
|
+-- ¿En qué formato viaja y cuántas veces se decodifica?
|
+-- ¿Se transforma, concatena, divide o normaliza?
|
+-- ¿Quién interpreta el resultado?
|   +-- parser SQL
|   +-- shell
|   +-- parser de opciones/argv
|   +-- URL/cliente HTTP
|   +-- filesystem/include
|   +-- máquina de estados
|
+-- ¿Qué gramática entiende y cuál es la sintaxis envolvente?
|
+-- ¿Qué capacidad mínima quiero demostrar?
|
+-- ¿Qué evidencia la distinguirá de caché, error o coincidencia?
|
+-- ¿Control positivo y negativo se separan de forma reproducible?
|   +-- no --> reducir, medir y reconsiderar la hipótesis
|   +-- sí --> documentar primitiva; adaptar una restricción cada vez

```

Cinco casos completamente desarrollados

Caso 1: contexto SQL numérico, no textual

GET /product?id=12 produce internamente SELECT name FROM products WHERE id = 12.

Payload incorrecto: ' OR '1'='1. Introduce una comilla en un lugar numérico y puede causar solo error. Payload correcto para la hipótesis: 12 AND 1=1, comparado con 12 AND 1=2.

Contexto de entrada: query id. Sintaxis original: WHERE id = <entrada>. Entrada controlada: expresión tras =. Transformaciones conocidas: URL decode una vez. Parser final: SQL del motor del laboratorio. Sink: ejecución de consulta. Primitiva: construir un oráculo booleano. Payload mínimo: 12 AND 1=1. Significado: conserva el ID y añade una condición verdadera. Resultado esperado: mismo producto; la variante 1=2 no devuelve fila. Control negativo: 999999 y 12 AND 1=2. Restricción: las comillas se rechazan. Por qué falla la variante básica: asume literal textual inexistente. Hipótesis de adaptación: contexto numérico. Payload adaptado: pareja verdadera/falsa sin comillas. Por qué debería funcionar: ambas expresiones son sintácticamente válidas y solo cambia el valor lógico. Evidencia: diferencia reproducible con cuerpo estable. Cuándo no funcionaría: conversión estricta a entero, consulta parametrizada o evaluación fuera de SQL.

Resolución: observación, una comilla causa 500; qué sé, solo hay un error; hipótesis, SQL textual, SQL numérico o validación; experimento, pareja numérica verdadera/falsa; resultado esperado, solo SQL concatenado conserva la diferencia; resultado obtenido, respuestas separadas; conclusión, oráculo booleano numérico; siguiente paso, identificar motor antes de usar funciones específicas.

Caso 2: ; id no demuestra una shell

La aplicación "comprueba un host". 127.0.0.1; id aparece dentro de ping: unknown host.

Payload incorrecto: repetir separadores. Payload correcto: primero provocar un error de operando y obtener argv mediante un wrapper de laboratorio.

Contexto: JSON {"host": "..."}.

Sintaxis original posible: H1 sh -c "ping -c 1 \$host"; H2 execve("ping", ["ping", "-c", "1", host]).

Entrada controlada: valor host.

Transformaciones: JSON decode.

Parser final: desconocido.

Sink: shell o ping.

Primitiva: distinguir gramática de shell de argumento literal.

Payload mínimo: 127.0.0.1; printf MARK.

Resultado esperado: H1 produce MARK; H2 entrega el texto a ping.

Control negativo: 127.0.0.1 y 127.0.0.1; printf NOPE con salida capturada.

Restricción: ; aparece en el error.

Por qué falla la variante básica: no existe shell o el valor está citado.

Hipótesis de adaptación: analizar argv y opciones del programa, no buscar otro separador.

Payload adaptado: una opción inocua solo si hay evidencia de un argumento adicional.

Evidencia: log argv o error inequívoco del parser de opciones.

Cuándo no funcionaría: un único elemento argv sin splitting o allowlist estricta.

Resolución: el error de ping prueba llegada a ping; el experimento muestra un único argv[3]; se descarta command injection y solo se mantiene como hipótesis una alteración de operandos si el wrapper divide la entrada.

Caso 3: filtro de ruta antes de la segunda decodificación

/download?name=..%2Fsecret.txt se bloquea; %252e%252e%252fsecret.txt intenta abrir un fichero fuera de la base.

Contexto: query name. Sintaxis original: open(base + "/" + name). Entrada controlada: nombre. Transformaciones: URL decode del framework, filtro, decode del router interno, normalización de ruta. Parser final: filesystem. Sink: open(). Primitiva: seleccionar un fichero fuera de base. Payload mínimo: un nivel .. / hacia un fichero marcador del laboratorio. Significado: .. selecciona padre; / separa segmento. Resultado esperado: contenido del marcador. Control negativo: fichero inexistente al mismo nivel. Restricción: filtro ve .. / tras la primera decodificación. Por qué falla la variante básica: la forma peligrosa existe durante la validación. Hipótesis de adaptación: una segunda capa decodifica después. Payload adaptado: percent-encoding de los signos %, solo para probar esa hipótesis. Por qué debería funcionar: la primera decodificación aún no produce separadores completos; la segunda sí. Evidencia: log de ambas formas y contenido inequívoco. Cuándo no funcionaría: validación sobre ruta canónica final, una sola decodificación o identificadores indirectos.

Conclusión inicial equivocada: "el WAF bloquea traversal". El log muestra que el bloqueo pertenece a la aplicación y que el router interno vuelve a decodificar; atribuirlo al WAF habría dirigido la adaptación a la capa equivocada.

Caso 4: callback que no era SSRF

Un campo avatar_url provoca una petición al listener, pero solo cuando se abre el perfil en Chrome.

Contexto: JSON de perfil. Sintaxis original: . Entrada controlada: atributo URL. Transformaciones: JSON decode, escape HTML, parser HTML del navegador. Parser final: navegador, no cliente HTTP del servidor. Sink: carga de subrecurso client-side. Primitiva buscada: demostrar quién inicia la conexión. Payload mínimo: URL única al listener. Resultado esperado por SSRF: callback al guardar, desde infraestructura del servidor. Control negativo: guardar sin abrir el perfil y consultar mediante cliente sin render. Restricción: callback depende de renderizado. Por qué falla la hipótesis básica: se atribuyó al servidor una petición del navegador. Hipótesis de adaptación: no procede; se descarta SSRF. Evidencia: IP, User-Agent y correlación temporal con Chrome. Cuándo cambiaría la conclusión: callback durante el guardado o desde el backend aun sin navegador.

Caso 5: PowerShell -> JSON -> shell remota

Se prueba un laboratorio que recibe JSON y, en una versión vulnerable, concatena query en sh -c "grep -n \"\$query\" data.txt".

Payload incorrecto: enviar comillas de shell sin conservar JSON, lo que causa 400 antes del sink. Payload correcto: construir primero el fragmento interno, después serializarlo como cadena JSON y finalmente protegerlo para PowerShell.

Contexto de entrada: cuerpo JSON enviado desde PowerShell. Sintaxis original: grep -n "<entrada>" data.txt. Entrada controlada: interior de comillas dobles de shell. Transformaciones: quoting local, bytes HTTP, JSON decode, concatenación, parsing Bash. Parser final: Bash. Sink: ejecución de comando. Primitiva: cerrar el argumento, ejecutar printf MARK y neutralizar el sufijo dentro del laboratorio. Payload mínimo interno: "; printf MARK; #. Significado: " cierra el argumento; ; separa; printf produce marcador; # comenta el sufijo. Resultado esperado: MARK una vez. Control negativo: JSON válido con MARK literal y una variante con separador dentro de comillas. Restricción: el primer intento recibe 400 invalid JSON. Por qué falla: el parser exterior nunca entrega el valor. Hipótesis de adaptación: serializar desde dentro hacia fuera. Payload adaptado: generar el objeto con ConvertTo-Json en vez de escapar manualmente. Por qué debería funcionar: PowerShell produce JSON válido que decodifica al fragmento previsto. Evidencia: captura de bytes, log del valor decodificado y marcador. Cuándo no funcionaría: ejecución directa sin shell, quoting seguro, allowlist o salida no capturada.

Comparación: payload correcto e incorrecto

Situación	Incorrecto	Problema	Correcto como experimento
SQL numérico	' OR '1'='1	inventa un literal textual	pareja 12 AND 1=1 / 12 AND 1=2
argv directo	; id, luego && id	cambia caracteres sin cambiar hipótesis	observar argv o error de opción inocua
JSON exterior	fragmento shell pegado a mano	puede morir en JSON	serializar objeto y verificar bytes
SSRF dudosa	apuntar a localhost	no demuestra quién solicita	listener único y control sin navegador
Ruta codificada	probar encodings al azar	no localiza la decodificación	variar una capa y registrar forma final

Reconocer la herramienta

Los mensajes de uso, opciones, versión, cabeceras y formato de error son huellas. Busca la frase exacta en documentación oficial, confirma versión y reproduce localmente con datos inocuos. No copies una opción de otra versión ni una sintaxis de un wrapper diferente.

Casos de construcción progresiva

SQL injection

Contexto: `SELECT * FROM items WHERE id = <entrada>`; entrada numérica. Primitiva: cambiar una condición. Construcción: conserva primero `id=7`; compara una expresión verdadera y una falsa válidas para el motor. Si ambas respuestas son iguales, no añadas extracción: comprueba caché, conversión a entero y si el valor llega a SQL. En contexto de texto, primero habría que cerrar el literal; en numérico no. Confirmación: diferencias reproducibles y controladas, no solo un error. Defensa: parámetros y allowlist para identificadores.

Command injection

Contexto interno hipotético: `sh -c "ping -c 1 <host>"`. Primitiva: id en laboratorio. El payload interno `127.0.0.1`; id contiene operando válido, separador de shell y segundo comando. Al enviarlo como JSON, las comillas exteriores pertenecen a JSON, no a la shell. Si `;` se bloquea, pregunta si otra gramática ya aceptada permite secuencia; no pruebes separadores al azar. Si la app usa `execve(["ping", "-c", "1", host])`, cambia la hipótesis a argumentos. Defensa: evitar shell, validar host semánticamente y fijar argumentos.

Argument y option injection

Contexto: la app ejecuta `curl <url>` mediante lista de argumentos. `;` id no se interpreta. Primitiva: provocar una opción inocua reconocida por esa versión. Construye una entrada que el wrapper divida en más de un argumento solo si existe evidencia de splitting. Un error `unknown option` demuestra llegada al parser de opciones, no ejecución. Defensa: un valor por elemento, `--` si está soportado, allowlist de esquema/destino y API que no reinterprete texto.

Path traversal

Contexto: `open('/srv/images/' + filename)`. Primitiva: leer un archivo conocido del laboratorio. Construye `../` por nivel desde la base conocida y normaliza mentalmente. Si se elimina una secuencia, compara original, filtrado y ruta canónica; una variante codificada solo tiene sentido si alguna capa decodifica después del filtro. Confirmación: contenido inequívoco o error que revele ruta resuelta, con control de archivo inexistente. Defensa: IDs indirectos, allowlist y comprobación de ruta canónica dentro de la base.

LFI

Contexto: `include(page + '.php')`. A diferencia de traversal de lectura, `include` puede interpretar contenido. Empieza con una vista válida, prueba ruta conocida y observa si el sufijo se añade. Los wrappers PHP solo aplican si PHP y esa configuración los soportan. El comando interno `curl -s '$URL/?page=../../../../etc/passwd'` transporta una ruta; las comillas protegen `$URL` en la shell local si son literales, por lo que en práctica debe adaptarse correctamente. Defensa: mapa fijo de identificador a plantilla.

SSRF

Contexto: el servidor obtiene la URL enviada. Primitiva: callback HTTP a un listener propio. El playbook usa `curl -sS '$URL/fetch?url=http://ATTACKER_IP:8000/':` primero sustituye variables y confirma la petición. Después compara loopback y destino inexistente solo en el laboratorio. Ante filtro, estudia parser, resolución y redirecciones; no asumas que otra escritura representa el mismo destino. Defensa: allowlist de destinos, resolución y validación de IP efectiva, bloqueo de redes no permitidas y revalidación tras redirección.

File upload

Capas: nombre declarado, Content-Type, bytes, nombre almacenado, ruta pública y handler del servidor. Primitiva: subir un archivo de texto marcador y recuperarlo. Luego verifica qué capa decide extensión/tipo. El ejemplo interno `curl -sS -F 'file=@shell.phtml;type=image/png' '$URL/upload'` mezcla bytes de un fichero con un tipo declarado; no demuestra por sí solo que se ejecute. Confirmación: recuperación, ubicación y, solo después, interpretación controlada. Defensa: nombre aleatorio, almacenamiento fuera del webroot, inspección de contenido y servidor sin ejecución.

SSTI

Contexto: entrada insertada en una plantilla evaluada. Primitiva: expresión aritmética inocua. `{{7*7}}` solo tiene significado para ciertas sintaxis; 49 reflejado confirma evaluación si el control literal no produce lo mismo. Después identifica motor y sandbox mediante mensajes/documentación antes de ampliar. Defensa: no compilar plantillas controladas y pasar datos como valores.

Autenticación

No siempre hay "cadena payload". Contexto: transición login -> MFA -> sesión autorizada. Primitiva: demostrar que el servidor acepta una transición no válida, por ejemplo manipulando un parámetro cuyo estado debería ser servidor. Cambia una variable, conserva sesión y compara acceso directo al recurso. Una pantalla omitida no confirma bypass si el backend rechaza la acción. Defensa: máquina de estados en servidor, rotación de sesión y autorización por petición.

Escalada de privilegios

El payload se construye desde una relación de permisos. Ejemplo sudo: salida de `sudo -l ->` regla concreta -> consulta de comportamiento documentado del binario -> prueba inocua -> capacidad efectiva. Ejemplo PATH: proceso privilegiado invoca nombre sin ruta -> directorio escribible precede al legítimo -> ejecutable marcador -> evidencia de identidad efectiva. No empieces con una reverse shell. Defensa: rutas absolutas, entorno controlado, privilegio mínimo y archivos no escribibles por identidades inferiores.

Anatomía obligatoria al documentar un payload

Contexto de entrada:
Sintaxis original:
Primitiva:
Payload interno:
Serialización de transporte:
Significado de cada componente:
Resultado esperado:
Control negativo:
Restricción observada:
Adaptación y motivo:
Evidencia final:

Errores frecuentes

- Escapar para la shell local y creer que se escapó para el parser remoto.

- Añadir varias técnicas a la vez y perder causalidad.
- Confundir reflexión con ejecución.
- Confundir error con confirmación.
- Copiar una variante de otro motor, versión o sistema operativo.
- ampliar impacto antes de demostrar la primitiva.
- usar callbacks fuera del entorno autorizado.

Resumen

Un payload correcto une contexto, gramática, transformación, sink y evidencia. La adaptación no consiste en cambiar caracteres hasta obtener respuesta, sino en probar una explicación sobre el procesamiento.

Chuleta operativa

1. Entrada y formato.
2. Sintaxis envolvente.
3. Parser final.
4. Primitiva mínima.
5. Serialización exterior.
6. Resultado y control negativo.
7. Una variación.
8. Clasificar fallo.
9. Adaptar con motivo.
10. Registrar evidencia y mitigación.

Referencias

- [Web Security Academy: materiales](#)
- [GNU Bash Reference Manual](#)
- [Conceptos internos](#)
- [Metadatos internos de comandos](#)

Anterior: [Metodología de laboratorio](#). Siguiendo: módulos de 03_seguridad_web/.